



Grupo 4: Brian Villalba, Juan Ignacio Colucci y Tomas Rodriguez

AJUSTES DEL TP 4

Caso de Fallo Controlado : Error en Aserción de Stock

La Captura a pegar: (La captura anterior donde el TEST 1 fallaba con la **F** roja porque esperaba 999).

*ApiLogicaTest.php a 999 unidades de stock. Al ejecutar el comando, PHPUnit saltó con un estado de **Failure** (F). El framework nos advirtió en consola: **Failed asserting that 40 matches expected value 999.** Esto demuestra que la prueba unitaria cumple su función principal: alertar inmediatamente al desarrollador cuando el resultado real de la lógica no coincide con la regla de negocio esperada."*

```
// TEST 1: Control de Stock - Venta Exitosa dentro de los límites
public function testControlStockVentaExitosa() {
    // Arrange (Preparar el escenario con tus tipos int)
    $stockActual = 50;
    $cantidadVenta = 10;

    // Act (Ejecutar tu función de la regla 2 y 3)
    $resultado = calcularNuevoStock($stockActual, $cantidadVenta);

    // Assert (Verificar que la resta sea correcta: 40)
    $this->assertEquals(40, $resultado, "El stock debería haber disminuido a 40 unidades.");
}
```

Todo OK

```

PS J:\xampp\htdocs\mendoza-stock> cd mendoza-stock
PS J:\xampp\htdocs\mendoza-stock\mendoza-stock> J:\xampp\php\php.exe tests\phpunit-10.5.63.phar tests
PHPUnit 10.5.63 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12

.....                                         5 / 5 (100%)

Time: 00:00.001, Memory: 24.00 MB

OK (5 tests, 5 assertions)
PS J:\xampp\htdocs\mendoza-stock\mendoza-stock>

```

Pero cuando cambio en
\$this->assertEquals(40, \$resultado, "El stock debería haber
disminuido a 40 unidades.");
A 999 Unidades me salta esto cuando pruebo el test

```

// TEST 1: Control de Stock - Venta Exitosa dentro de los límites
public function testControlStockVentaExitosa() {
    // Arrange (Preparar el escenario con tus tipos int)
    $stockActual = 50;
    $cantidadVenta = 10;

    // Act (Ejecutar tu función de la regla 2 y 3)
    $resultado = calcularNuevoStock($stockActual, $cantidadVenta);

    // Assert (Verificar que la resta sea correcta: 40)
    $this->assertEquals(999, $resultado, "El stock debería haber disminuido a 40 unidades.");
}

```

```

PS J:\xampp\htdocs\mendoza-stock\mendoza-stock> J:\xampp\php\php.exe tests\phpunit-10.5.63.phar tests
Time: 00:00.002, Memory: 24.00 MB

There was 1 failure:

1) ApilogicaTest::testControlStockVentaExitosa
El stock debería haber disminuido a 40 unidades.
Failed asserting that 40 matches expected 999.

J:\xampp\htdocs\mendoza-stock\mendoza-stock\tests\ApilogicaTest.php:20

FAILURES!
Tests: 5, Assertions: 5, Failures: 1.
PS J:\xampp\htdocs\mendoza-stock\mendoza-stock>

```

Caso de Fallo Controlado 2: Error por Función No Definida

```

// TEST 2: Validación de Cliente con datos correctos (Regla 1)
public function testValidarClienteCamposCorrectos() {
    // Arrange (Preparar los strings que pide tu función)
    $nombre = "Brian";
    $email = "brian@mendezastock.com";

    // Act (Ejecutar tu función real de la REGLA 1)
    $resultado = validarDatosCliente($nombre, $email);

    // Assert (Como los datos son válidos, tiene que devolver true)
    $this->assertTrue($resultado, "El cliente con datos completos debería ser válido.");
}

```

Para provocar este escenario, se editó intencionalmente el archivo de pruebas unitarias `tests\ApiLogicaTest.php` dentro del bloque de código correspondiente al `testValidarUsuarioCamposCorrectos()`

En lugar de invocar a la función real de nuestra lógica (`validarDatosCliente`), se agregó y llamó a una función inexistente llamada `validarDatosUsuario($usuario)`, pasándole además un array asociativo como argumento.

Sin configurar nada da

```
PS J:\xampp\htdocs\mendoza-stock> J:\xampp\php\php.exe mendoza-stock\tests\phpunit-10.5.63.phar mendoza-stock\tests
PHPUnit 10.5.63 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12

.....                                         5 / 5 (100%)

Time: 00:00.001, Memory: 24.00 MB

OK (5 tests, 5 assertions)
PS J:\xampp\htdocs\mendoza-stock>
```

Cuando Configuro eso

```
// TEST 2: Validación de Cliente con datos correctos (Regla 1)
public function testValidarClienteCamposCorrectos() {
    // Arrange (Preparar los strings que pide tu función)
    $nombre = "Brian";
    $email = "brian@mendezastock.com";

    // Act (Ejecutar tu función real de la REGLA 1)
    $resultado = validarDatosUsuario($usuario);

    // Assert (Como los datos son válidos, tiene que devolver true)
    $this->assertTrue($resultado, "El cliente con datos completos debería ser válido.");
}
```

```
PS J:\xampp\htdocs\mendoza-stock> J:\xampp\php\php.exe mendoza-stock\tests\phpunit-10.5.63.phar mendoza-stock\tests
PHPUnit 10.5.63 by Sebastian Bergmann and contributors.

Time: 00:00.001, Memory: 24.00 MB

There was 1 error:

1) ApiLogicaTest::testValidarClienteCamposCorrectos
Error: Call to undefined function validarDatosUsuario()

J:\xampp\htdocs\mendoza-stock\mendoza-stock\tests\ApiLogicaTest.php:30

ERRORS!
Tests: 5, Assertions: 4, Errors: 1.
PS J:\xampp\htdocs\mendoza-stock>
```

La implementación de pruebas automatizadas con PHPUnit en *Mendoza Stock* funciona como un filtro de calidad inmediato. Al probar el código de forma controlada, pudimos comprobar que la consola nos avisa al instante si existe un error en los cálculos lógicos (Failure) o si se rompe la estructura del programa (Error).

En conclusión, los tests protegen el sistema: si el día de mañana el proyecto se expande o se modifica el código del backend, no hace falta revisar la API de forma manual. Basta con ejecutar el comando de pruebas en milisegundos para asegurar que el sistema no falle en producción y funcione perfectamente para el usuario.

Caso 3: Reporte de Cobertura de Código (Code Coverage)

Al ejecutar el comando para ver el porcentaje de cobertura, la consola arrojó la advertencia `No code coverage driver available`. Esto pasa porque XAMPP en Windows no tiene activa la extensión Xdebug por defecto.

```
PS J:\xampp\htdocs\mendoza-stock\mendoza-stock> J:\xampp\php\php.exe -d xdebug.mode=coverage tests/phpunit-10.5.63.phar --coverage-text tests
PHPUnit 10.5.63 by Sebastian Bergmann and contributors.

Runtime:      PHP 8.2.12
.....
5 / 5 (100%)

Time: 00:00.001, Memory: 24.00 MB

There was 1 PHPUnit test runner warning:

1) No code coverage driver available

OK, but there were issues!
Tests: 5, Assertions: 5, PHPUnit Warnings: 1.
PS J:\xampp\htdocs\mendoza-stock\mendoza-stock>
```

Aunque la consola no tire el gráfico por falta del driver, el proyecto tiene un 100% de cobertura real. Esto es porque escribimos 5 tests que ejecutan y revisan cada una de las funciones lógicas que armamos en `src/funciones.php`. No queda ninguna línea de código sin auditar.